

Advanced Hybrid Scoring for ATS

As a Data Scientist building a Hybrid Search Engine, directly combining BM25 and Dense Vector scores is tricky. This document outlines why your model feels "less sensitive," how to fix it, and the industry-standard advanced methods.

1. The Core Problem: Unbounded vs. Bounded Scales

If you are just doing: $\text{Final Score} = (\alpha * \text{BM25}) + ((1 - \alpha) * \text{Cosine})$ without normalizing first, your math is broken.

- **Cosine Similarity (Dense):** Always outputs a bounded score, typically between **0.0** and **1.0**. Because LLMs understand context well, most resumes will score between 0.6 and 0.85. It is a very "compressed" scale.
- **BM25 (Sparse):** Outputs an **unbounded** score (e.g., 2.4, 15.6, 54.2) based on term frequency and document length.

The Result: If a candidate gets a BM25 score of 25.0, it will completely obliterate the Cosine

score of 0.8, rendering your α weight useless. The model loses sensitivity because BM25 is bullying the math.

The Fix: Min-Max Normalization

Before applying your alpha weight, you *must* normalize the BM25 scores of your candidate pool to a 0.0 - 1.0 scale.

Python snippet for normalization

```
def min_max_normalize(scores):  
    min_score = min(scores)  
    max_score = max(scores)  
    if max_score == min_score:  
        return [0.5 for _ in scores]  
    return [(s - min_score) / (max_score - min_score) for s in scores]
```

```
normalized_bm25 = min_max_normalize(raw_bm25_scores)
```

```
# Now you can safely apply your alpha!
```

```
hybrid_score = (alpha * normalized_bm25[i]) + ((1 - alpha) * dense_scores[i])
```

2. What is the "Ultimate" Alpha (α) Threshold?

There is no single magic number, but in the HR Tech industry, systems generally favor exact

keyword matches over semantic matches.

- **Recommended Baseline:** $\alpha = 0.6$ to 0.7 (Where Alpha is the BM25 weight).
- **Why?** If a job requires "React.js", a candidate with "Vue.js" will get a high Semantic score (they are both frontend frameworks). However, a recruiter explicitly wants React. A higher alpha (e.g., 0.7) ensures that the exact keyword match acts as an anchor, while the semantic score (0.3) acts as a "bonus" for candidates who use the right conceptual language.

3. Advanced Method 1: Reciprocal Rank Fusion (RRF)

If tuning alpha and normalizing scores feels too finicky, the industry standard for Hybrid Search (used by Elasticsearch and Pinecone) is **Reciprocal Rank Fusion (RRF)**.

Instead of combining their *raw scores*, RRF combines their *rankings*.

- Candidate A is Rank #1 in BM25, and Rank #4 in Semantic.
- Candidate B is Rank #3 in BM25, and Rank #2 in Semantic.

The Math:

$$RRFScore = \frac{1}{k + Rank_{BM25}} + \frac{1}{k + Rank_{Dense}}$$

(Note: k is a smoothing constant, usually set to 60).

Why it's better: It completely bypasses the problem of scale mismatch. A candidate must consistently rank high in both exact matching and conceptual matching to win.

4. Advanced Method 2: The Two-Stage Pipeline (Cross-Encoders)

If your Dense (Semantic) score feels "less sensitive" (i.e., every candidate gets a 0.75 cosine similarity and it's hard to tell them apart), you are hitting the limit of **Bi-Encoders** (standard SentenceTransformers).

Bi-encoders compress the JD and the Resume separately into vectors. This loses deep contextual nuance. To fix this, use a **Cross-Encoder** for re-ranking.

The Ultimate Architecture:

1. **Stage 1 (Retrieval):** Use BM25 + Bi-Encoder to filter 1,000 resumes down to the Top 50.
2. **Stage 2 (Re-Ranking):** Pass the Job Description and the Top 50 resumes simultaneously

into a Cross-Encoder.

The Cross-Encoder reads both texts *together*, word by word, and outputs a highly sensitive relevance score.

```
# python snippet for src/scoring/dense_vector.py
from sentence_transformers import CrossEncoder

# Load a pre-trained cross encoder
model = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

job_description = "Seeking a Python developer with API experience..."
resumes = ["Resume 1 text...", "Resume 2 text..."]

# Pair the JD with each resume
pairs = [[job_description, res] for res in resumes]

# The model outputs highly sensitive relevance scores
highly_sensitive_scores = model.predict(pairs)
```

Summary for your Streamlit App:

Start by applying **Min-Max Normalization** to your BM25 scores in Python before you apply the slider's alpha. If it still doesn't feel right, implement **RRF**—it requires zero parameter tuning and is incredibly robust!